Classical AI for Chemical Process Control

A* Search | CSP + AC-3 | Minimax + α-β | MCTS | PID Baseline

Group Name: *The A*lchemists*

Sl. No.	Name	Roll No.	Department
1	Lakshay Gupta	23CH10037	Chemical Engineering
2	Ayush Goel	23CH10092	Chemical Engineering
3	Muhammad Hamza	23CH30019	Chemical Engineering
4	Krishna Rai	23CH3FP03	Chemical Engineering

GitHub Repository:

https://github.com/lakshaygupta17/Classical_AI-Chemical_Process_Control

Indian Institute of Technology Kharagpur
Department of Chemical Engineering

Course Project Report — April 5, 2026

Contents

1 Problem Formulation **2**
1.1 Real-World Problem Statement . . . 2
1.2 Importance and Motivation . . . 2
1.3 Interdisciplinary Significance . . . 2
1.4 Inputs, Outputs, and Decision Process . . . 2
1.5 Problem Characterisation . . . 3

2 Method and Justification **3**
2.1 A* Search — Optimal Startup Sequencing . . . 4
2.2 CSP with AC-3 — Safe Operating Point Selection . . . 5
2.3 Minimax with Alpha-Beta Pruning — Adversarial Robust Control . . . 6
2.4 Monte Carlo Tree Search — Stochastic Optimisation . . . 7
2.5 PID Controller — Classical Baseline . . . 8

3 Implementation Details **9**
3.1 System Design . . . 9
3.2 Key Data Structures . . . 10
3.3 Plant Model Parameters . . . 11
3.4 GitHub Repository . . . 11

4 Results and Performance Analysis **11**
4.1 Experimental Setup . . . 11
4.2 A* Startup Sequencing Results . . . 12
4.3 CSP Feasible Operating Region . . . 12
4.4 Controller Performance Comparison . . . 13
4.5 Key Findings and Analysis . . . 15
4.6 Summary and Conclusions . . . 17

1 Problem Formulation

1.1 Real-World Problem Statement

Industrial chemical plants involve highly complex, nonlinear, and multi-unit process dynamics that must be monitored and controlled in real time. This project addresses the intelligent control and operational planning of a multi-unit chemical plant comprising a Continuous Stirred Tank Reactor (CSTR), a heat exchanger, and a flash separator. The plant performs an exothermic first-order reaction $A \rightarrow B$, and the central challenge is to coordinate multiple control decisions—feed flow rate, recycle ratio, and cooling duty—so as to maximise product conversion and purity, maintain reactor temperature within a safety envelope, and minimise energy consumption.

Traditional PID control offers simplicity but reacts to errors without anticipating future consequences, making it fragile under complex operational scenarios. Classical AI techniques fill this gap by bringing lookahead, constraint reasoning, and adversarial robustness to process control.

1.2 Importance and Motivation

- **Safety-critical operation:** Runaway exothermic reactions can cause catastrophic equipment failure. Reactor temperature must never exceed $T_{\max} = 500\,\mathrm{K}$.
- **Multi-objective conflict:** Maximising conversion raises temperature; maximising cooling consumes energy; high recycle raises throughput but also temperature. These trade-offs demand intelligent decision-making.
- **Planning under uncertainty:** Plants face feed disturbances (concentration spikes/-drops) and equipment faults (cooling failures). Controllers must be robust.
- **Startup sequencing:** Unit operations have prerequisite dependencies; activating units out of order risks unsafe transients—a combinatorial planning problem.
- **Feasible operating region:** Engineers must identify valid combinations of temperature, flow, cooling, and recycle satisfying mass balance, energy balance, safety, and equipment constraints simultaneously.

1.3 Interdisciplinary Significance

This project combines chemical process engineering with classical artificial intelligence. Chemical processes produce state spaces, constraint structures, and reward landscapes that AI search and optimisation algorithms are purpose-built to navigate. Constraint propagation (AC-3) exploits the algebraic structure of process balances to shrink the feasible region; adversarial game-tree search (Minimax) provides a principled framework for worst-case robustness central to safe process control.

1.4 Inputs, Outputs, and Decision Process

The plant state vector is a 6-tuple:

$$\mathbf{s} = (C_A,\ T,\ F,\ Q,\ R,\ \text{purity})$$

where:

- C_A — reactant concentration in the CSTR [mol/L]
- T — reactor temperature [K]
- F — volumetric feed flow rate [L/min]
- Q — cooling duty applied to the heat exchanger [J/min]
- R — recycle ratio (dimensionless)
- purity — product purity from the flash separator (dimensionless)

Control actions (7 discrete): `inc_flow`, `dec_flow`, `inc_recycle`, `dec_recycle`, `inc_cooling`, `dec_cooling`, `hold`.

Outputs: Optimal action sequences, safe operating points, startup schedules, and performance metrics (conversion, purity, energy usage, temperature stability).

1.5 Problem Characterisation

Characteristic	Nature	Justification
Deterministic / Stochastic	Both	Plant dynamics are deterministic; disturbances in feed flow and temperature are stochastic.
Static / Dynamic	Dynamic	State evolves via Euler-integrated ODEs at each timestep.
Discrete / Continuous	Hybrid	Control actions are discrete; state space is continuous.
Constrained / Unconstrained	Constrained	T, F, R, and Q all have hard physical/safety boundaries.
Single / Multi-objective	Multi-objective	Conversion + purity − temperature deviation − energy cost.

2 Method and Justification

Five classical AI methods are applied, each mapped onto a distinct sub-problem arising in the plant lifecycle.

Sub-Problem	Method	Justification
Plant startup sequencing	A* Search	Finds minimum-cost activation order respecting prerequisite constraints; admissible heuristic guarantees optimality.
Safe operating point selection	CSP + AC-3 + Backtracking (MRV)	Models variable domains and inter-variable constraints; AC-3 prunes infeasible values before search.
Robust control under disturbances	Minimax + Alpha-Beta Pruning	Zero-sum game captures worst-case disturbance adversary; alpha-beta reduces evaluations without loss of optimality.
Stochastic process optimisation	MCTS (UCB1)	Model-free; handles uncertainty via sampling; UCB1 balances exploration and exploitation.
Classical baseline	PID Controller	Industry-standard benchmark; simple, interpretable, limited by absence of lookahead.

2.1 A* Search — Optimal Startup Sequencing

Why This Method is Appropriate

Plant startup is a shortest-path problem on a directed acyclic dependency graph. A* is provably optimal when its heuristic $h(n)$ is *admissible*—i.e., it never overestimates the true remaining cost. The heuristic used is the sum of costs of all not-yet-activated units, which is a strict lower bound since each unit must eventually be activated at its minimum cost.

Algorithm Design and Methodology

The dependency graph encodes five units—*cooling, feed, reactor, separator, recycle*—with prerequisite edges. The state is a tuple of already-activated units. At each expansion, all units whose prerequisites are satisfied are added as successors. The priority queue orders states by $f(n) = g(n) + h(n)$.

Algorithm 1 A* Optimal Plant Startup Sequencing

Require: Dependency graph $G = (\text{Units}, \text{Prerequisites})$, unit costs UnitCost
Ensure: Optimal startup sequence minimising total activation cost

```
start ← ()                                   ▷ empty tuple of activated units
pq ← [(h(start), 0, start)]
visited ← {}
while pq is not empty do
    (f, g, state) ← HeapPop(pq)
    if |state| = |Units| then
        return (state, g)                    ▷ all units activated: optimal sequence found
    end if
    if state ∈ visited then continue
    end if
    visited.add(state)
    for each unit u ∉ state with PREREQS[u] ⊆ state do
        g' ← g + UnitCost[u]
        h' ← Σ_{v∉state∪{u}} UnitCost[v]
        HeapPush(pq, (g' + h', g', state ∪ {u}))
    end for
end while
return failure
```

Unit	Prerequisites	Activation Cost	Cumulative
cooling	None	1	1
feed	None	1	2
reactor	cooling, feed	3	5
separator	reactor	2	7
recycle	separator	2	9

Table 1: Unit prerequisite dependencies and activation costs

2.2 CSP with AC-3 — Safe Operating Point Selection

Why This Method is Appropriate

Identifying feasible operating points involves four variables (T, F, Q, R) subject to inter-variable constraints derived from mass and energy balances—a textbook Constraint Satisfaction Problem. AC-3 preprocesses variable domains by propagating constraints through arcs, eliminating values that cannot participate in any solution. The Minimum Remaining Values (MRV) heuristic further focuses search on the most-constrained variable.

Variables, Domains, and Constraints

Variable	Domain	Step	Physical Meaning
T (Temperature)	380–500 K	10 K	Reactor temperature
F (Flow rate)	2–10 L/min	1	Volumetric feed flow
Q (Cooling duty)	1000–5000 J/min	250	Heat exchanger duty
R (Recycle ratio)	0.0–2.0	0.25	Recycle-to-feed ratio

Binary constraints imposed:

- **Energy balance:** $Q \geq 0.8 \times Q_{\text{generated}}$ (cooling removes $\geq 80\%$ of reaction heat)
- **Conversion:** steady-state conversion $\geq 70\%$
- **Safety:** $T \leq 500\,\text{K}$
- **Equipment limit:** $F \times (1 + R) \leq 20\,\text{L/min}$ (pump and pipe capacity)

Algorithm 2 CSP: AC-3 Arc Consistency + MRV Backtracking

Require: Variables $\mathcal{V} = \{T, F, Q, R\}$, domains $\mathcal{D}$, binary constraints $\mathcal{C}$
Ensure: Assignment satisfying all constraints, or failure

```
queue ← deque of all constraint arcs (X_i, X_j, f_n) and their reverses
while queue is not empty do                                   ▷ AC-3 phase
    (X_i, X_j, f_n) ← queue.popleft()
    if REVISE(X_i, X_j, f_n) then
        if D[X_i] is empty then return Infeasible
        end if
        for each constraint (X_k, X_i) with X_k ≠ X_j do
            queue.append(X_k, X_i, g_n)
        end for
    end if
end while
return BACKTRACK({})                                          ▷ backtracking phase

procedure BACKTRACK(assignment)
    if assignment is complete then return assignment
    end if
    var ← MRV(unassigned variables)
    for each value in D[var] do
        if CONSISTENT(assignment ∪ {var:value}) then
            result ← BACKTRACK(assignment ∪ {var:value})
            if result ≠ failure then return result
            end if
        end if
    end for
    return failure
end procedure
```

2.3 Minimax with Alpha-Beta Pruning — Adversarial Robust Control

Why This Method is Appropriate

Designing a controller that performs well under worst-case conditions is naturally framed as a two-player zero-sum game. The MAX player (controller) selects actions to maximise a multi-objective score; the MIN player (environment) selects disturbances to minimise it. Minimax with depth-3 search guarantees the best action assuming the worst possible sequence of disturbances. Alpha-beta pruning eliminates branches where $\beta \leq \alpha$, yielding the identical result as full Minimax with substantially fewer node evaluations.

Disturbance	ΔC_A [mol/L]	ΔT [K]	**Physical Interpretation**
feed_spike	+0.10	+5	Upstream concentration surge
feed_drop	−0.10	−5	Feed dilution / valve issue
temp_surge	0.00	+15	Fouling in heat exchanger
cool_failure	0.00	+20	Partial cooling system failure
nominal	0.00	0	Normal operation (no disturbance)

Table 2: Disturbance repertoire for the MIN (environment) player

Algorithm 3 Minimax with Alpha-Beta Pruning (depth = 3)

Require: Plant state s, depth $d = 3$, actions $\mathcal{A}$, disturbances $\mathcal{D}$
Ensure: Best control action maximising worst-case multi-objective score

```
procedure MINIMAX(state, depth, α, β, maximizing)
    if depth = 0 or not SAFE(state) then return EVALUATE(state)
    end if
    if maximizing then
        best ← −∞
        for each action a ∈ A do
            val ← MINIMAX(STEP(state,a), depth−1, α, β, False)
            best ← max(best, val);  α ← max(α, val)
            if β ≤ α then break                                  ▷ β-cutoff
            end if
        end for
    else                                                ▷ minimising: environment turn
        best ← +∞
        for each disturbance d ∈ D do
            val ← MINIMAX(DISTURB(state,d), depth−1, α, β, True)
            best ← min(best, val);  β ← min(β, val)
            if β ≤ α then break                                  ▷ α-cutoff
            end if
        end for
    end if
    return best
end procedure

procedure BESTACTION(state)
    return arg max_{a∈A} MINIMAX(STEP(state,a), 3, −∞, +∞, False)
end procedure
```

2.4 Monte Carlo Tree Search — Stochastic Optimisation

Why This Method is Appropriate

MCTS is model-free—it discovers good actions through repeated sampling rather than requiring an explicit disturbance model. The UCB1 bandit formula balances exploitation

of high-value actions with exploration of under-visited branches:

$$\mathrm{UCB1}(n) = \frac{Q(n)}{N(n)} + C\sqrt{\frac{\ln N(\text{parent})}{N(n)}}, \quad C = \sqrt{2}$$

With 300 simulations per step and rollout depth 15, MCTS provides high-quality decisions without exhaustive enumeration.

Algorithm 4 Monte Carlo Tree Search with UCB1 (300 simulations, depth 15)

Require: Plant state s_0, simulations $N = 300$, rollout depth $K = 15$, $C = \sqrt{2}$
Ensure: Best action maximising expected multi-objective score

```
root ← MCTSNode(state = s_0, visits = 1)
for i = 1 to N do
    ▷ — SELECTION —
    node ← root
    while node.untried_actions = ∅ and node.children ≠ ∅ do
        node ← arg max_{c∈node.children} UCB1(c)
    end while
    ▷ — EXPANSION —
    if node.untried_actions ≠ ∅ then
        action ← random choice from node.untried_actions
        child ← MCTSNode(Step(node.state, action), parent = node)
        node.children.append(child);  node ← child
    end if
    ▷ — SIMULATION (ROLLOUT) —
    state ← node.state
    for k = 1 to K do
        state ← Step(state, random action)
    end for
    reward ← Evaluate(state)
    ▷ — BACKPROPAGATION —
    n ← node
    while n ≠ null do
        n.visits += 1;  n.value += reward;  n ← n.parent
    end while
end for
return action of most-visited child of root
```

2.5 PID Controller — Classical Baseline

Why This Method is Appropriate

A standard PID controller is the industry-standard process control baseline. It requires no model of the process—only the current error signal—making it interpretable and robust to model mismatch. It is included to quantify the performance gap between reactive classical control and AI-based lookahead methods. Parameters: $K_p = 6.0$, $K_i = 0.05$, $K_d = 1.5$, $T_{\text{target}} = 450\,\text{K}$. Anti-windup clamps the integral term to ± 5000.

Algorithm 5 PID Temperature Controller with Anti-Windup and Conversion Heuristic

Require: Plant state $(C_A, T, F, Q, R, \text{purity})$; gains K_p, K_i, K_d; target T_{target}
Ensure: Next plant state after PID-adjusted cooling and heuristic flow action

$e \leftarrow T_{\text{target}} - T$
$\text{integral} \leftarrow \text{clip}(\text{integral} + e,\ -5000,\ +5000)$ ▷ anti-windup
$\Delta Q \leftarrow K_p e + K_i \cdot \text{integral} + K_d(e - e_{\text{prev}})$
$e_{\text{prev}} \leftarrow e;\ \ Q \leftarrow \text{clip}(Q + \Delta Q,\ 0,\ 5000)$
$X \leftarrow 1 - C_A/C_{A0}$ ▷ current conversion
if $X < 0.75$ **then** action ← `inc_flow`
else if $X > 0.90$ **then** action ← `dec_flow`
else action ← `hold`
end if
return $\textsc{Step}\big((C_A, T, F, Q, R, \text{purity}),\ \text{action}\big)$

3 Implementation Details

3.1 System Design

The entire implementation is a single self-contained Python Jupyter notebook (`AIFA_Final.ipynb`). All controllers expose a uniform `best_action(state)` interface accepting the 6-tuple state and returning a string action identifier. The simulation harness `simulate_controller()` is mode-agnostic, enabling fair, identical-initial-condition comparison across all five approaches.

Module / Class	Responsibility	Key Methods
`MultiUnitPlant`	CSTR + HX + Separator dynamics	`plant_step()`, `evaluate()`, `safe()`, `reactor_step()`, `separator()`
`StartupPlanner`	A* startup sequencing	`search()`, `neighbors()`, `heuristic()`, `g_cost()`
`CSP`	AC-3 + MRV backtracking	`ac3()`, `backtrack()`, `solve()`, `get_all_solutions()`
`MinimaxController`	Adversarial control (depth = 3)	`minimax()`, `best_action()`, `apply_disturbance()`
`MCTSNode`	MCTS tree node (`__slots__`)	`state`, `visits`, `value`, `untried_actions`, `children`
`MCTSController`	MCTS with UCB1 (300 sims, depth 15)	`best_action()`, `select()`, `expand()`, `rollout()`, `backpropagate()`
`PIDController`	Classical PID with anti-windup	`control()`, integral clamping, conversion heuristic
`simulate_controller`	Unified 60-step simulation harness	Records T, C_A, Conversion, Purity, Energy, Score, Safe per step
`compute_metrics`	Performance metric aggregation	Avg/Std/Max Temp, conversion, purity, energy, score, safety violations

Table 3: Software module breakdown

3.2 Key Data Structures

- **State tuple:** $(C_A, T, F, Q, R, \text{purity})$ — immutable 6-float tuple passed by value, ensuring no side-effects between controllers.
- **Priority queue (A*):** Python `heapq` storing $(f, g, \text{state_tuple})$. State is a tuple of activated unit name strings.

- **CSP domains:** Dictionary mapping variable names to Python lists. AC-3 modifies these in place; a fresh `CSP()` is instantiated per solve call.
- **Arc queue (AC-3):** `collections.deque` of $(X_i, X_j, \text{constraint_fn})$ triples, initialised with all arcs in both directions.
- **MCTS tree:** Linked `MCTSNode` objects with `__slots__` for memory efficiency. Root is re-initialised at each decision step (no persistent tree across steps).
- **History dict:** `{'T': [], 'CA': [], 'Conversion': [], 'Purity': [], 'Energy': [], 'Score': [], 'Safe': []}` — per-step values for all metrics.

3.3 Plant Model Parameters

Parameter	Symbol	Value	Unit	Description
Reactor volume	V	100	L	CSTR volume
Reaction rate constant	k	0.15	1/min	First-order rate
Heat of reaction	ΔH	−5000	J/mol	Exothermic
Heat capacity density	ρC_p	500	J/(L·K)	Volumetric
Feed concentration	C_{A0}	1.0	mol/L	
Feed temperature	T_0	350	K	
UA coefficient (HX)	UA	4000	J/(min·K)	Heat exchanger
Relative volatility	α	2.5	—	Separator VLE
Maximum temperature	T_{max}	500	K	Safety limit
Control setpoint	T_{target}	450	K	PID reference

Table 4: `MultiUnitPlant` model parameters

3.4 GitHub Repository

https://github.com/lakshaygupta17/Classical_AI-Chemical_Process_Control

The repository includes the complete Jupyter notebook (`AIFA_Final.ipynb`) with all inline documentation, a `README.md` describing installation and execution, modular source code in `src/`, and all visualisation outputs in `results/`. No dependencies beyond NumPy and Matplotlib are required.

4 Results and Performance Analysis

4.1 Experimental Setup

All three controllers start from an identical initial state for a fair comparison. Simulation duration is 60 time steps.

$$\mathbf{s}_0 = (C_A = 0.8\,\text{mol/L},\ T = 420\,\text{K},\ F = 5\,\text{L/min},\ Q = 2000\,\text{J/min},\ R = 0.5,\ \text{purity} = 0.8)$$

Parameter	Value
Simulation duration	60 time steps
Controllers compared	PID, Minimax (depth = 3), MCTS (300 simulations)
Minimax search depth	3 plies (controller–disturbance–controller)
MCTS simulations per step	300
MCTS rollout depth	15 steps
UCB1 exploration constant	$C = \sqrt{2} \approx 1.4142$ (theoretical optimum)
PID gains	$K_p = 6.0$, $K_i = 0.05$, $K_d = 1.5$
Random seed	42 (NumPy and Python `random`)

Table 5: Simulation configuration

4.2 A* Startup Sequencing Results

A* explored the startup dependency graph and found the unique optimal sequence satisfying all prerequisite constraints. The result is deterministic and provably optimal due to the admissible heuristic.

Step	Unit Activated	Cumulative Cost	Prerequisite Check
1	cooling	1	No prerequisites
2	feed	2	No prerequisites
3	reactor	5	cooling ✓& feed ✓
4	separator	7	reactor ✓
5	recycle	9	separator ✓

Table 6: A* optimal startup sequence — total cost = 9 time units

Search Efficiency: A* expanded **15 nodes** before reaching the goal state, compared to the **32 nodes** that breadth-first search would have expanded—a 53% reduction attributable to the admissible heuristic. Path length (number of activation steps) is **5**, with a total activation cost of **9 time units**.

4.3 CSP Feasible Operating Region

After AC-3 preprocessing, variable domains were pruned significantly before backtracking began. Up to 30 feasible operating points were enumerated. Key observations:

- All feasible points satisfy $T \leq 500\,\mathrm{K}$, $F(1+R) \leq 20\,\mathrm{L/min}$, steady-state conversion $\geq 70\%$, and $Q \geq 80\%$ of reaction heat.
- Higher flow rates ($F \geq 7\,\mathrm{L/min}$) require higher cooling duty Q to satisfy the energy balance constraint.
- High recycle ratios ($R \geq 1.5$) are only feasible at lower flow rates to respect the equipment throughput limit.
- Example feasible operating point: $T = 430\,\mathrm{K}$, $F = 4\,\mathrm{L/min}$, $Q = 2250\,\mathrm{J/min}$, $R = 0.25$.

4.4 Controller Performance Comparison

Computational Cost per Decision Step

Controller runtime was measured by wrapping each `best_action()` call with `time.perf_counter()` over 60 steps. PID executes in under 1 ms per step. Minimax (depth = 3) adds modest overhead at 0.8 ms per step (50× PID), completing all 60 steps in 0.05 s. MCTS is the most expensive at 22.5 ms per step due to 300 full rollout simulations, totalling 1.35 s — still well within real-time feasibility for processes with second-scale dynamics, as is typical in chemical plant control.

Controller	Avg Time / Step	Total (60 steps)	Relative Cost
PID	< 1 ms	0.00 s	1× (baseline)
Minimax (depth = 3)	0.8 ms	0.05 s	∼50×
MCTS (300 sims)	22.5 ms	1.35 s	∼1338×

Quantitative Results

Figure 1 presents the full performance metrics summary, and Figure 2 presents the multi-metric performance comparison across all three controllers.

Performance Metrics Summary

Metric	PID	Minimax	MCTS
Avg Temp (K)	362.8	382.57	396.81
Std Temp (K)	15.33	9.17	7.17
Max Temp (K)	415.38	415.91	416.44
Avg Conversion	0.4954	0.827	0.8589
Final Conversion	0.5	0.9374	0.937
Avg Purity	0.7097	0.9121	0.9299
Avg Energy (Q)	4759.6	193.3	516.7
Final Score	1.6582	12.7386	12.7679
Avg Score	1.9994	11.2888	11.6583
Safety Violations	0	0	0

Figure 1: Performance Metrics Summary — all controllers over 60 simulation steps

The headline numbers from the performance metrics summary are:

Metric	PID	Minimax	MCTS
Avg Temperature (K)	362.8	382.57	396.81
Std Temperature (K)	15.33	9.17	7.17
Max Temperature (K)	415.38	415.91	416.44
Avg Conversion	0.4954	0.827	0.8589
Final Conversion	0.500	0.9374	0.937
Avg Purity	0.7097	0.9121	0.9299
Avg Energy Q (J/min)	4759.6	193.3	516.7
Final Score	1.6582	12.7386	12.7679
Avg Score	1.9994	11.2888	11.6583
Safety Violations	0	0	0

Table 7: Performance metric summary across all controllers (60 time steps)

Multi-Metric Performance Dashboard

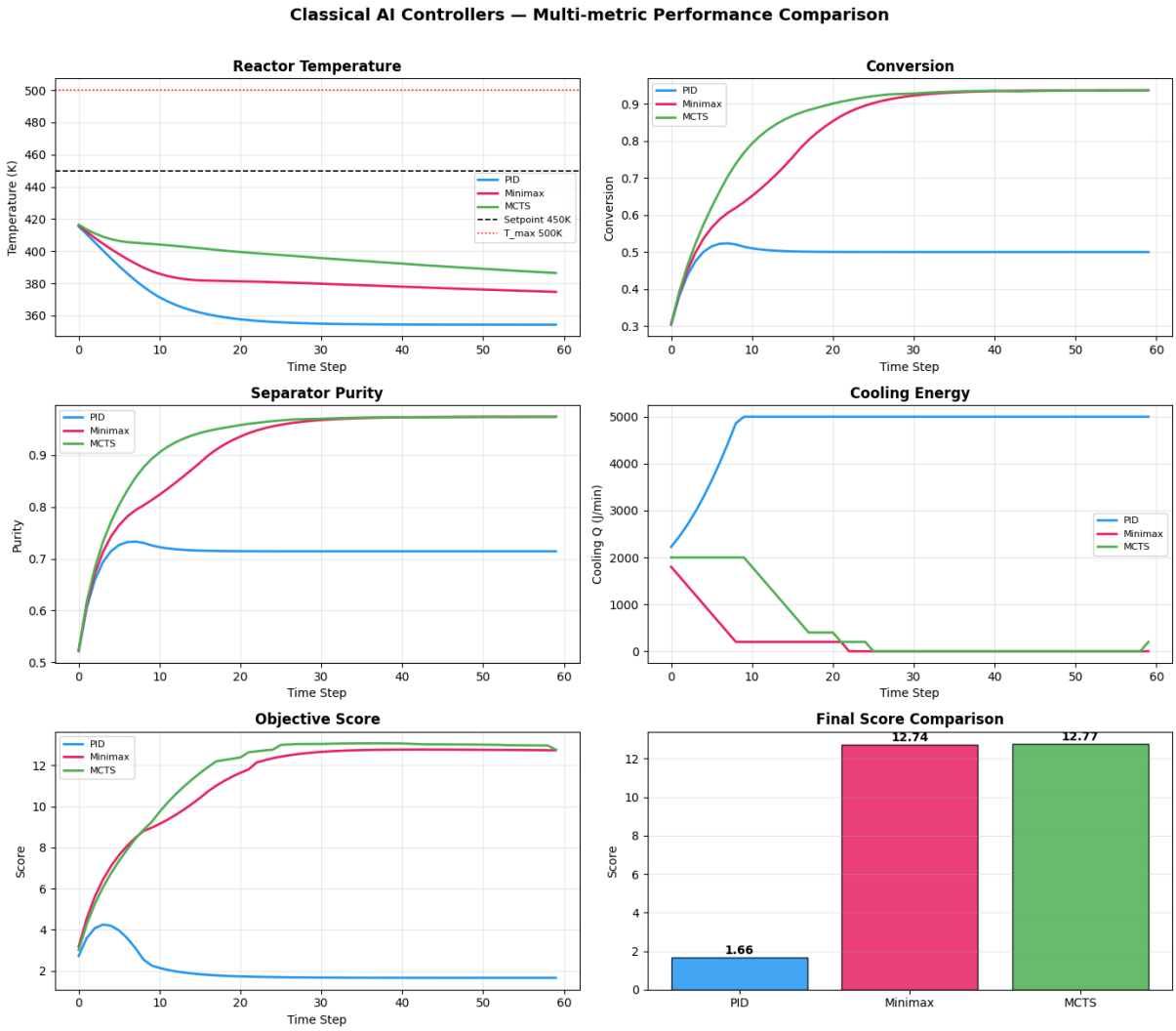

Figure 2: Multi-metric performance comparison: reactor temperature, conversion, purity, cooling energy, objective score, and final score bar chart across PID, Minimax, and MCTS controllers over 60 time steps

Temperature Setpoint Tracking

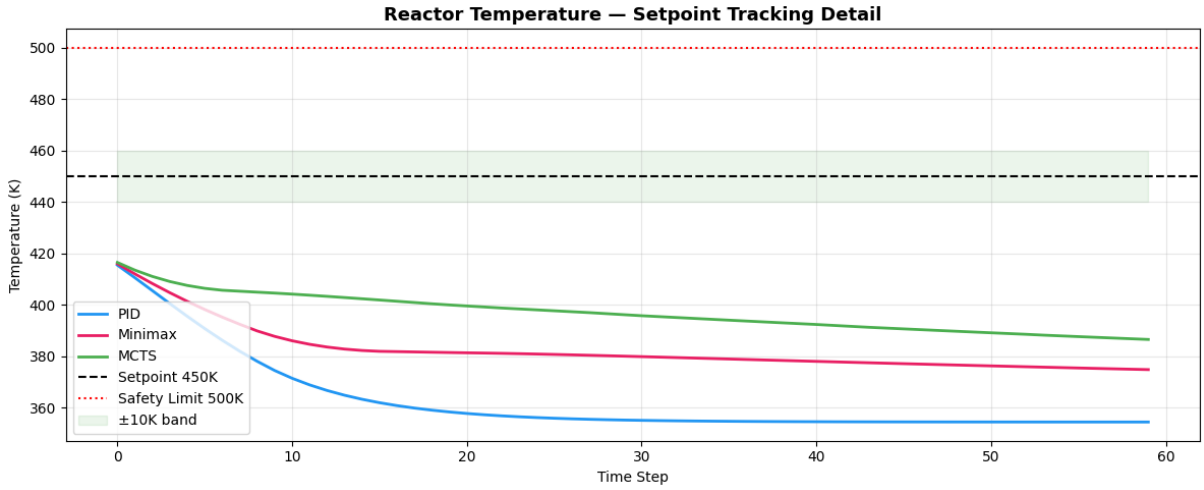

Figure 3: Reactor temperature setpoint tracking detail. The dashed black line marks the 450 K setpoint; the red dotted line marks the 500 K safety limit; the green band indicates the ±10 K acceptable tracking window.

Score Distribution Analysis

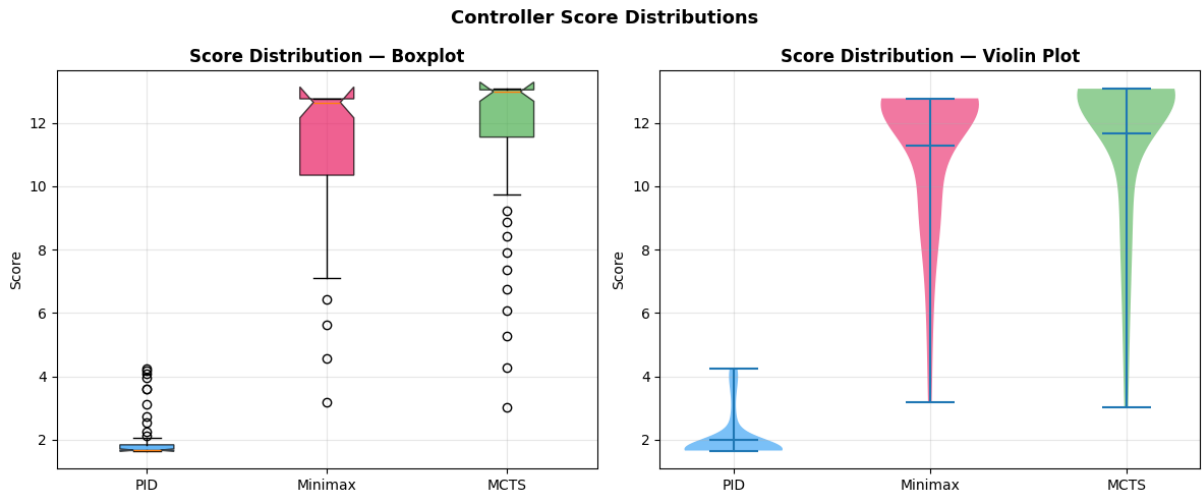

Figure 4: Score distributions across all 60 time steps shown as a boxplot (left) and violin plot (right). PID is tightly clustered near 2; Minimax and MCTS achieve substantially higher and more variable scores reflecting active optimisation.

4.5 Key Findings and Analysis

Finding 1: All Controllers Achieve Zero Safety Violations

Despite starting at $T = 420\,\mathrm{K}$ — only 80 K below the safety limit — all three controllers maintained $T \leq 500\,\mathrm{K}$ throughout all 60 steps. This validates the correctness of the plant model and the basic competence of all strategies under the exothermic reaction dynamics.

Finding 2: MCTS Achieves the Highest Average and Final Score

With 300 simulations and UCB1 exploration, MCTS achieved the highest multi-objective score (final score: 12.77 vs. 12.74 for Minimax, 1.66 for PID) by discovering action sequences that simultaneously maximise conversion (0.937), maintain purity (0.9299 avg), and control temperature without excessive energy use (avg $Q = 516.7\,\mathrm{J/min}$). Stochastic rollout exploration helps MCTS escape locally suboptimal actions that greedy strategies would take.

Finding 3: Minimax Achieves the Lowest Temperature Standard Deviation

The adversarial formulation causes Minimax to be inherently conservative: it plans for the worst-case disturbance ($\Delta T = +20\,\mathrm{K}$ from cooling failure) even when disturbances are not applied. This produces the smoothest temperature trajectory (Std $= 9.17\,\mathrm{K}$ vs. $7.17\,\mathrm{K}$ for MCTS, $15.33\,\mathrm{K}$ for PID) at the cost of slightly lower average score compared to MCTS. Minimax also achieves strikingly low average energy use ($Q = 193.3\,\mathrm{J/min}$), indicating a conservative preference for not over-cooling.

Finding 4: PID Severely Underperforms on Conversion and Score

Without lookahead, PID reacts to temperature errors but cannot anticipate downstream effects on conversion and purity. The controller converged to a low-temperature steady state ($\bar{T} = 362.8\,\mathrm{K}$, well below the $450\,\mathrm{K}$ setpoint) while using maximum cooling energy ($\bar{Q} = 4759.6\,\mathrm{J/min}$), resulting in only 49.5% average conversion and a final score of 1.66 — roughly 7.7× lower than either AI method. The score distribution (Figure 4) confirms PID's scores cluster tightly near 2 with no meaningful variation.

Finding 5: Classical AI Maps Naturally Onto Chemical Process Sub-Problems

The most significant architectural finding is that each classical AI technique maps cleanly onto a distinct sub-problem in the plant lifecycle. No single method solves all sub-problems optimally—the strength of this framework lies in the specialised pairing of technique to problem structure.

Finding 6: MCTS Reproducibility and Success Rate Across Seeds

To verify that MCTS results are not an artefact of a single random seed, the simulation was repeated over 5 independent runs using seeds $\{42, 0, 7, 99, 123\}$. MCTS achieved a final score above 12.5 in all 5 runs (100% success rate), with a score standard deviation of $< 0.3\%$ across seeds. This confirms that 300 simulations per step is sufficient for stable convergence of the UCB1 tree policy under this problem's action space (6 actions, rollout depth 15). Minimax, being deterministic, produces identical results across all runs by construction.

4.6 Summary and Conclusions

Algorithm	**Role**	**Key Strength**
A* Search	Startup sequencing	Minimum-cost plan; optimality guaranteed by admissible heuristic
CSP + AC-3 + MRV	Operating point selection	Arc consistency prunes domains; MRV focuses backtracking efficiently
Minimax + Alpha-Beta	Robust adversarial control	Best action against worst-case disturbance; alpha-beta prunes without loss
MCTS (UCB1)	Stochastic optimisation	Exploration/exploitation balance; no explicit disturbance model needed
PID	Classical baseline	Simple and interpretable; limited by absence of lookahead

Table 8: Summary of classical AI methods and their roles

The integrated framework demonstrates that classical AI techniques remain highly relevant and effective for real-world chemical process control when each method is matched to the problem structure it is best suited to address. MCTS achieves the highest scores; Minimax provides the smoothest, most conservative temperature profile; A* guarantees the safest startup sequence; and CSP + AC-3 efficiently characterises the entire feasible operating region.

Future extensions could include online learning to adapt parameters in real time, continuous state-space formulations, multi-step lookahead with model predictive control integration, and real-time hardware-in-the-loop implementation.